Preventing race conditions is a crucial aspect of using `useEffect` effectively, especially when dealing with asynchronous operations. A race condition can occur when multiple asynchronous operations complete in an unexpected order, leading to inconsistent state or incorrect behavior. To mitigate this, it is recommended to implement a cleanup function within `useEffect`. This function executes when the component unmounts or when the dependencies change, allowing any pending asynchronous operations—such as API requests—to be canceled.

For example, a common approach involves using a boolean flag to check if the component is still mounted before updating the state with fetched data. If the component has unmounted, the state update is skipped, preventing memory leaks or errors. Alternatively, the `AbortController` API can be used to cancel ongoing fetch requests cleanly. Following these practices helps ensure React components remain performant, robust, and reliable when handling complex side effects.

Figure A.4 Flowchart of React functional component lifecycle phases, from mounting to unmounting, with associated hooks